personal-v4 Application Review

> Laravel 12 + Filament v5 — All-in-One Personal Dashboard
> **Author:** Nova Ardiansyah

Summary

A comprehensive personal dashboard application that combines finance management, debt tracking, email, blogging, file management, uptime monitoring, gallery, notes, short URLs, notification systems (Telegram + Push), and activity tracking—all managed through a Filament SPA admin panel with a REST API.

---

1. Existing Features

1.1 Finance & Payment System (`app/Models/Payment*`, `app/Filament/Resources/Payments/`)

Status: ✅ **Most Mature Module**

- Income, Expense, Transfer, and Withdrawal with automatic balance mutation for Payment Accounts
- Payment Accounts (many-to-many transfers)
- Payment Categories (including `is_default`)
- Draft payments with approval workflow (balances remain unchanged until approval)
- Scheduled payments processed automatically via queue
- Itemized transactions (Items + Pivot with quantity, price, and total)
- Financial overview reports (daily, weekly, monthly)
- Reports for total balance, scheduled expenses, and drafts
- PDF report generation via mPDF
- Excel export using Laravel Excel (maatwebsite)

- Dashboard widgets with expense charts by category and statistics overview
- Soft deletes across all related models

1.2 Debt Management (`app/Models/Debt*`)

Status: ✅ Solid New Feature

- Debt tracking including lender, principal, admin fee, and disbursement
- Interest rate and service fee support
- Annuity payment calculations
- Automatic installment generation based on loan tenure
- Installment statuses: unpaid / paid
- Payment statuses: ongoing / partial_payment / paid
- Automatically creates draft Payment records when installments are paid
- Integrated with Payment Accounts

1.3 Blog System (`app/Models/Blog*`)

Status: ✅ Mature

- Blog posts with categories and tags
- Draft, Published, and Scheduled statuses
- Cover image upload
- SEO fields (meta title & description)
- Subscriber management with automatic email notifications
- Polymorphic author management
- Scheduled publishing

1.4 Email System (`app/Models/Email*`)

Status: ✅ Mature

- Queue-based email sending
- Email templates with placeholder system
- Optional header and footer
- Polymorphic file attachments
- Draft → Sent workflow
- Email preview in the admin panel

- Automatic login notification emails (separate templates for Web and API)
- Unique UID tracking

1.5 Uptime Monitoring (`app/Models/UptimeMonitor*`)

Status: ✅ Mature

- HTTP endpoint monitoring
- Response time tracking
- Status classification: UP / DOWN / SLOW (300ms threshold)
- Queue-based automatic monitoring with configurable intervals
- Telegram notifications for status changes
- HTTP Status Code reference table
- Detailed logs for every check

1.6 Gallery (`app/Models/Gallery`)

Status: ✅ Mature

- Image uploads with polymorphic relationships
- Automatic image optimization (300px, 900px, 1600px, original)
- Image size classification
- Group code support
- Public/private visibility
- Soft deletes

1.7 Notes (`app/Models/Note`)

Status: ✅ Simple & Functional

- CRUD operations
- Pin and archive functionality
- Polymorphic file attachments
- Soft deletes

1.8 Short URLs (`app/Models/ShortUrl`)

Status: ✅ Feature Complete

- URL shortening with custom domain support
- QR Code generation (HeroQR)
- Click tracking
- Notes for each short URL
- File download integration

1.9 File Management (`app/Models/File`, `app/Models/FileDownload`)

Status: ✅ Mature

- Polymorphic file attachments
- Temporary signed download URLs (1-month expiration)
- File aliases
- File size tracking
- Scheduled automatic deletion
- Download statistics

1.10 Contact Messages (`app/Models/ContactMessage`)

Status: ✅ Simple

- Public API endpoint for contact forms
- Read/unread status
- Soft deletes
- Telegram and Email reply triggers

1.11 Push Notifications (`app/Models/PushNotification`)

Status: ✅ **Functional**

- Expo Push Notification integration
- Per-user notification tokens
- Enable/disable notifications
- Delivery success/failure tracking

1.12 Telegram Notifications (`app/Services/TelegramService`)

Status: ✅ **Mature**

- Full Telegram Bot integration
- Notifications for login, uptime, debt, payment, and activities
- Event-driven and Job-based dispatching

1.13 Activity Logging (`app/Models/ActivityLog`)

Status: ✅ **Mature**

- Resource-level activity logging (Create, Update, Delete)
- Device information tracking (IP, country, city, region, geolocation, timezone, user agent, referer)
- Polymorphic causer and subject
- Email preview directly from activity logs

1.14 Authentication & User (`app/Models/User`, `app/Services/AuthService`)

Status: ✅ **Mature**

- Multi-factor authentication (Authenticator App + Email OTP)
- Sanctum API tokens
- Avatar upload
- Login notifications via Telegram and Email with deduplication interval
- Login IP geolocation (ipinfo.io)
- Soft deletes

1.15 Code Generation (`app/Models/Generate`)

Status: ✅ Powerful Utility

- Sequential code generation per model or alias
- Format: prefix + YYMM + sequence (0001–9999) + suffix
- Monthly auto-reset
- Supports multiple models (Payments, Short URLs, Debts, etc.)

1.16 Settings (`app/Models/Setting`)

Status: ✅ Utility

- Cached key-value configuration
- Stores application settings such as:
  - Short URL domain
  - Telegram chat ID
  - Notification intervals
  - Default system user

1.17 Skills (`app/Models/Skill`)

Status: ✅ Simple

- Skill management for portfolio/showcase

---

2. API Routes (`routes/api/`)

Status: ✅ Well Organized

- Authentication (login, logout, token validation, password change, profile update)
- Payment CRUD, reports, and PDF generation
- Payment Accounts
- Payment Types
- Payment Goals

- Goal Statuses
- Items & Item Types
- Notes
- Blog (Posts, Categories, Tags, Subscribers)
- Gallery
- Short URLs
- Contact Messages
- Notification Settings & Testing
- Push Notifications
- Code Generation
- Health Check Endpoint

---

3. Areas for Improvement

3.1 Code Consistency

Issue	Details	Recommendation
Mixed static & instance methods	`Payment::mutateDataPayment()` is static but sometimes called as an instance	Standardize and move into a dedicated service
Duplicate balance mutation logic	Similar implementations across multiple methods	Extract into a single reusable service
Inline HTML in services	Notification HTML is hardcoded	Move templates into Blade views

---

3.2 Security

Issue	Details	Severity
Missing API rate limiting	Public APIs have no throttling	Medium
Missing CSRF protection on testing webhook	POST endpoint without CSRF middleware	Low
Hardcoded signed URL expiration	Fixed 1-month validity	Low
Ownership validation	Some queries lack `where('user_id', ...)`	Requires audit

3.3 Performance

Issue	Details	Recommendation
Potential N+1 queries	Separate balance calculations	Use joins or caching
Unlimited settings cache	`rememberForever()` without invalidation	Clear cache automatically on updates
Hardcoded chunk sizes	Queue/PDF chunk values are fixed	Make configurable

---

3.4 UX & Admin Experience

Issue	Recommendation
Search & Filters	Improve global search and filtering
Bulk Actions	Add bulk delete/export support
Validation	Apply `normalizeValidationErrors()` consistently

3.5 Testing Coverage

Based on the project structure, testing should include:

- Comprehensive feature tests for the Payment System
- Unit tests for Debt annuity calculations
- HTTP mock tests for Uptime Monitoring
- Integration tests for REST API endpoints

---

4. Feature Ideas

4.1 High Priority (High Value, Low Effort)

Feature	Description
Subscription Tracker	Track recurring subscriptions (Netflix, Spotify, etc.) with reminders

---

Feature	Description
Budget Planning	Monthly budget limits with spending alerts
Personal Dashboard 2.0	Fully customizable dashboard layout
Dark Mode	Enable Filament's built-in dark mode

4.2 Medium Priority

- API Key Management with scopes
- Outgoing Webhooks
- Habit Tracker
- Investment Portfolio
- Calendar / Planner
- OCR Receipt Scanner

4.3 Low Priority

- Two-way Telegram Bot
- Family Sharing
- Personal CRM
- AI-powered Payment Categorization
- Interactive API Documentation
- Time Tracking / Work Log

---

5. Architecture Review

Strengths

- Excellent use of polymorphic relationships (Files, Galleries, Activity Logs)
- Well-designed sequential code generation system
- Clear service layer separation
- Modern PHP 8 Attributes (`#[ObservedBy]`)
- Modular API route organization

Weaknesses

- Duplicate balance mutation logic
- Inline HTML and hardcoded strings in services
- Validation handled directly in controllers instead of Form Requests
- No automatic cache invalidation for Settings
- Limited automated test coverage for critical business logic

---

6. Conclusion

Overall, this is a well-structured application with personal finance as its strongest core module.

The application already provides a mature ecosystem consisting of multi-channel notifications, file management, activity logging, queue processing, and modern Laravel architecture patterns such as Observers, Service Classes, and Polymorphic Relationships.

Top Priorities

1. Refactor duplicate balance mutation logic.
2. Implement automatic cache invalidation for Settings.
3. Add API rate limiting and perform ownership validation audits.
4. Improve automated testing coverage for financial modules.

Quick Wins

- Recurring Subscription Tracker (extending scheduled payments)
- Monthly Budget Planning
- A more informative and customizable dashboard

Review generated on July 13, 2026, based on the project's code structure analysis.